

ESCUELA SUPERIOR POLITECNICA DE CHIMBORAZO

FACULTAD DE INFORMATICA Y ELECTRONICA

CARRERA DE TECNOLOGIAS DE LA INFORMACION

SISTEMA WEB DE MONITOREO Y ADMINISTRACION DE LA INFRAESTRUCTURA DE REDES - ESPOCH

ESTANDAR DE INGENIERIA: GESTION DE VERSIONES Y FLUJO GIT

Manual Tecnico de Trabajo Colaborativo, Roles de Ingenieria, Ciclo de Vida y Convenciones

Responsable del Proceso:

- Equipo de Desarrollo de Software
- Area de Aseguramiento de Calidad
- Coordinacion de Arquitectura TI

Comite Academico Evaluador:

- Infraestructura TI
- Tecnologia Web
- Conmutacion y Enrutamiento
- Big Data
- Diseno y Multimedia

Periodo Academico: Quinto Semestre (PAO 5)

Documento Oficial de Ingenieria de Software (Version 2.0 - Estandar Generico)

Orellana - Ecuador

Noviembre de 2026

Índice general

1. Metafora del Control de Versiones: El Patio Ferroviario **4**

1.1. Concepto de la Metafora: El Patio Ferroviario Central 4

2. Roles de Ingenieria y Matriz de Responsabilidades Tecnicas **6**

2.1. Responsabilidades Especificas por Rol Tecnico 6

2.1.1. Perfil: Lider Tecnico y Arquitecto Backend 6

2.1.2. Perfil: Desarrollador Frontend y Disenador UI/UX 7

2.1.3. Perfil: Especialista en Bases de Datos y Aseguramiento de Calidad (QA) 7

2.1.4. Perfil: Ingeniero DevOps, Redes e Infraestructura 7

3. Estrategia de Ramas y Ciclo de Vida del Software **8**

3.1. Estructura Jerarquica de Ramas 8

4. Estandar de Commits y Trazabilidad (Conventional Commits) **10**

4.1. Estructura Obligatoria del Mensaje 10

4.2. Tipos de Commit Permitidos 10

4.3. Reglas de Redaccion de Commits 10

5. Flujo de Trabajo Paso a Paso y Comandos Operativos **12**

5.1. Paso 1: Configuracion Inicial de la Identidad Local 12

5.2. Paso 2: Inicio de una Nueva Caracteristica (Branching) 12

5.3. Paso 3: Desarrollo Local y Commits Atomicos 12

5.4. Paso 4: Sincronizacion Continua contra Develop 13

5.5. Paso 5: Publicacion de la Rama y Creacion del Pull Request 13

5.6. Paso 6: Plantilla Obligatoria de Pull Request (Code Review) 13

5.7. Paso 7: Integracion y Limpieza de Ramas Locales 13

6. Politicas de Proteccion de Ramas y Estrategias de Fusion **14**

6.1. Politicas de Proteccion en Ramas Principales (`main` y `develop`) 14

6.2. Estrategias Estandar de Fusion en Pull Requests 14

7. Resolucion de Conflictos y Tecnicas Operativas Avanzadas 15

7.1. Protocolo Estandar de Resolucion de Conflictos 15

7.2. Gestion de Trabajo en Progreso con `git stash` 15

7.3. Deshacer Cambios con Seguridad: `git revert` frente a `git reset` 16

8. Versionado Semantico y Gestion de Etiquetas (Git Tagging) 17

8.1. Estandar SemVer 2.0.0 17

8.2. Comandos de Creacion y Gestion de Etiquetas 17

9. Automatizacion con Git Hooks y Diagnostico del Repositorio 18

9.1. Control Preventivo de Calidad con Git Hooks 18

9.2. Comandos de Diagnostico e Inspeccion del Historial 18

10. Seguridad del Repositorio y Gestion de Archivos Ignorados 19

10.1. Regla de Oro: Cero Secretos en el Repositorio 19

10.2. Archivo `.gitignore` Obligatorio 19

10.3. Plantilla de Variables de Entorno (`.env.example`) 20

11. Conclusiones 22

Índice de tablas

2.1. Matriz de asignacion de responsabilidades y prefijos por rol tecnico 6

4.1. Tipos estandar de commit y su aplicabilidad 11

Capítulo 1

Metafora del Control de Versiones: El Patio Ferroviario

1.1 Concepto de la Metafora: El Patio Ferroviario Central

Para estructurar el trabajo colaborativo concurrente en el equipo de desarrollo sin provocar colisiones de codigo, perdidas de trabajo ni ramas rotas, se adopta formalmente la metafora operativa de:

"La Red Ferroviaria y el Taller Central de Ensamblaje"

Bajo esta perspectiva conceptual:

- **La Via Principal de Pasajeros (`main`):** Representa la linea ferrea de alta velocidad en servicio comercial activo. Por aqui solo transitan trenes inspeccionados, certificados y listos para la demostracion ante los docentes, evaluadores y usuarios finales. Ningun desarrollador puede realizar modificaciones ni pruebas directamente sobre la via principal.
- **El Patio Central de Maniobras (`develop`):** Es la estacion de pruebas integradas donde convergen los vagones terminados. Todo nuevo modulo o funcionalidad se engancha primero aqui para verificar que la energia, las comunicaciones de red y los procesos se sincronicen armonicamente con el resto del sistema.
- **Los Talleres de Construccion Individual (`feature/*`):** Son vias laterales cerradas donde cada desarrollador ensambla un componente especifico (un motor analitico, una pantalla de interfaz, un daemon de sondeo o un validador de seguridad). En su taller, el desarrollador puede iterar, probar y reconstruir su pieza con total libertad sin afectar a los demas trenes en marcha.
- **La Inspeccion Tecnica Obligatoria (*Pull Request & Code Review*):** Antes de que un modulo nuevo salga de su via lateral hacia el patio central de maniobras, debe ser auditado formalmente por otro ingeniero del equipo mediante revision cruzada de codigo, verificando que la construccion cumpla los estandares de calidad, pruebas automatizadas y especificaciones del proyecto.

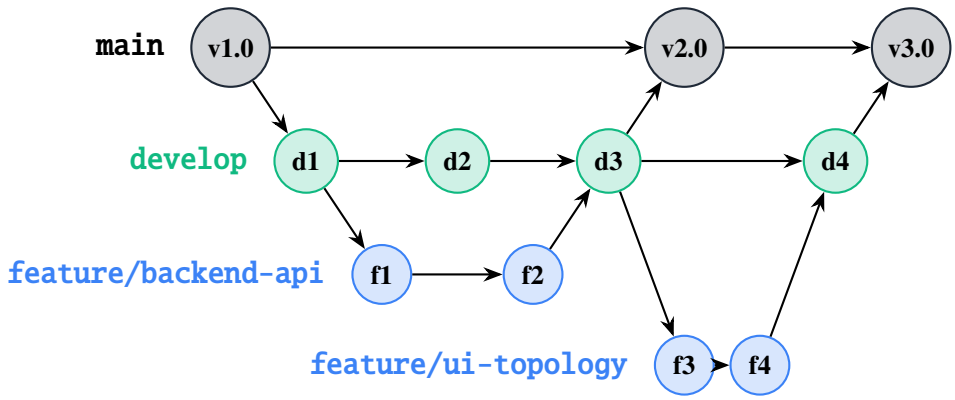


Figura 1.1: Topologia del modelo de ramas Git adaptado para el equipo

Capítulo 2

Roles de Ingenieria y Matriz de Responsabilidades Tecnicas

Para garantizar la mantenibilidad, escalabilidad y propiedad tecnica del codigo (*Code Ownership*), el proyecto distribuye las responsabilidades en cuatro perfiles de ingenieria complementarios. Cualquier desarrollador que se integre al proyecto asume uno de estos roles y sus compromisos asociados:

Tabla 2.1: Matriz de asignacion de responsabilidades y prefijos por rol tecnico

Rol de Ingenieria	Ambito Tecnico	Prefijos de Rama Autorizados
Lider Tecnico y Arquitecto Backend	Capa de negocio, API REST, seguridad JWT y daemon centinela	`feature/auth-*`, `feature/daemon-*`, `feature/api-config`
Desarrollador Frontend y Disenador UI/UX	Interfaz de usuario React, componentes WCAG y visualizacion	`feature/ui-*`, `feature/dashboard-*`, `feature/views-*`
Especialista en Bases de Datos y QA Lead	Esquemas NoSQL, indices TTL, pipelines analiticos y pruebas	`feature/db-*`, `feature/analytics-*`, `feature/qa-*`
Ingeniero DevOps y Redes	Servidor Debian 12, Nginx, socket de bajo nivel y despliegue	`feature/devops-*`, `feature/network-*`, `feature/infra-*`

2.1 Responsabilidades Especificas por Rol Tecnico

2.1.1 Perfil: Lider Tecnico y Arquitecto Backend

- **Gobernanza del Repositorio:** Administrador de las politicas del repositorio central. Responsable de activar las reglas de proteccion de ramas en `main` y `develop`, asegurando que ninguna operacion destructiva (`push -force`) reescriba el historial compartido.
- **Capa de Negocio y Seguridad (RF-01, RF-02, RF-05, RF-07, RF-10):** Construye y mantiene los endpoints REST en Express, la autenticacion dual con tokens JWT (Access Token de 15 min en memoria y Refresh Token de 8h en cookie segura HttpOnly), el control de acceso RBAC y el daemon planificador de disponibilidad.

- **Aprobacion de Pull Requests:** Revisa que cada integracion cumpla las pautas de arquitectura, manejo asincrono con `asyncHandler` y esquemas de validacion defensiva con Zod.

2.1.2 Perfil: Desarrollador Frontend y Disenador UI/UX

- **Arquitectura de la Interfaz Web (RF-08, RF-09, RF-12):** Implementa la aplicacion cliente en React con Vite, Tailwind CSS y la paleta institucional Mineral Green (#395649).
- **Visualizacion y Ergonomia Operativa (RNF-11):** Desarrolla el tablero NOC, la navegacion por facetas del inventario, la representacion de topologia y la linea temporal de eventos.
- **Accesibilidad Universal (WCAG 2.1 AA):** Garantiza la regla de triple codificacion en badges de estado (Color + Icono + Texto en mayusculas) para usuarios con daltonismo, y asegura el correcto contraste visual y navegabilidad por teclado.

2.1.3 Perfil: Especialista en Bases de Datos y Aseguramiento de Calidad (QA)

- **Persistencia y Modelado NoSQL (RF-03, RF-06, RF-11, RNF-08):** Disena los esquemas Mongoose para MongoDB, asegurando la consistencia entre colecciones, inmutabilidad de bitacoras y restricciones de unicidad.
- **Gestion de Ciclo de Vida de Telemetria (Seccion 8.4):** Configura los indices TTL de depuracion automatica (30 dias para comprobaciones brutas en `checks`, 90 dias para eventos en `events` y 24h para sesiones expiradas).
- **Control de Calidad (QA):** Supervisa que las tareas cumplan con la Definicion de Terminado (DoD) y valida los escenarios de prueba en formato Gherkin antes de otorgar el visto bueno para la fusion de cada Pull Request.

2.1.4 Perfil: Ingeniero DevOps, Redes e Infraestructura

- **Infraestructura y Servidor de Produccion (RNF-01, RNF-14, RNF-15):** Configura y administra la maquina con sistema operativo Debian 12 GNU/Linux, asegurando el proxy inverso Nginx, la segregacion de puertos y la ejecucion continua mediante servicios del sistema Systemd.
- **Sondeo de Red de Bajo Nivel (RF-04):** Desarrolla y empaqueta los modulos de ejecucion nativa de comandos seguros (`ping` y `arp-scan`) utilizando `child_process.spawn` con sanitizacion contra inyeccion de argumentos shell.
- **Entorno de Despliegue y Reproducibilidad:** Mantiene los scripts de arranque, el aprovisionamiento de dependencias del sistema y la verificacion de puertos abiertos y reglas de cortafuegos.

Capítulo 3

Estrategia de Ramas y Ciclo de Vida del Software

El proyecto implementa una variante pragmatica, rigurosa y estandarizada de **GitFlow**, adaptada para ciclos de entrega continuos en sprints:

3.1 Estructura Jerarquica de Ramas

1. **Rama `main` (Produccion y Entregable Oficial Estable):**
   - Aloja exclusivamente codigo estable, compilable, probado y validado que representa los cierres de hito del proyecto.
   - Todo commit en `main` debe estar etiquetado con un tag semantico formal (ejemplo: `v1.0.0`, `v2.0.0`).
   - **Regla estricta:** Nadie realiza commits o pushes directos a `main`. Los cambios unicamente ingresan mediante Pull Request desde `develop` al finalizar cada ciclo planificado.
2. **Rama `develop` (Integracion Continua de Desarrollo):**
   - Es el tronco principal de trabajo diario. Refleja el estado consolidado del sprint en curso con todas las historias de usuario aprobadas.
   - Sirve de punto de partida para cualquier nueva caracteristica y como destino de todos los Pull Requests de desarrollo.
   - Al igual que en `main`, los pushes directos estan bloqueados; la integracion se realiza a traves de revisiones de codigo.
3. **Ramas de Caracteristica (`feature/<modulo>-<nombre-descriptivo>`):**
   - Se crean siempre a partir de la version mas reciente de `develop`.
   - Tienen un ciclo de vida corto (idealmente de 1 a 3 dias de trabajo).
   - No deben contener trabajo mezclado de multiples requerimientos o tareas disjuntas.
   - **Ejemplos estandar:**
     - `feature/auth-jwt-login` (Implementacion de autenticacion)
     - `feature/ui-dashboard-kpis` (Tablero de control NOC)
     - `feature/db-event-immutability` (Coleccion inmutable de eventos)
     - `feature/daemon-icmp-ping` (Motor de sondeo periodico)
4. **Ramas de Correccion en Desarrollo (`bugfix/<modulo>-<descripcion>`):**

- Se desprenden de `develop` para solucionar fallos identificados durante las pruebas internas de integracion o revisiones de QA.
- Ejemplo: `bugfix/monitor-ping-timeout-calculation`.

5. **Ramas de Parche Critico (`hotfix/<version>`):**
   - Se originan directamente desde `main` exclusivamente ante una falla critica imprevista en el entregable oficial.
   - Una vez validada la correccion, se fusionan tanto en `main` como en `develop` para mantener la sincronizacion.
   - Ejemplo: `hotfix/v1.0.1-session-cookie-flags`.

Capítulo 4

Estandar de Commits y Trazabilidad (Conventional Commits)

Para garantizar la trazabilidad exigida por las normativas de calidad de software (ISO/IEC/IEEE 29148) y facilitar la generacion automatizada de bitacoras de cambio (*Changelog*), todos los commits deben adherirse estrictamente a la especificacion **Conventional Commits v1.0.0**.

4.1 Estructura Obligatoria del Mensaje

```
<tipo>(<alcance>): <descripcion concisa en imperativo> [#<
   id_requerimiento>]

[Cuerpo explicativo opcional: justificacion del cambio y decisiones
    tomadas]

[Pie opcional: referencias a tareas cerradas o notas de cambios
   incompatibles]
```

4.2 Tipos de Commit Permitidos

4.3 Reglas de Redaccion de Commits

- **Modo imperativo:** Redactar en tiempo presente (ej. “crear”, “corregir”, “configurar”), nunca en pasado (“creado”, “corregi”).
- **Primera letra en minuscula:** Despues de los dos puntos del prefijo, la descripcion comienza siempre con letra minuscula.
- **Sin punto final:** La linea principal del commit no debe concluir con punto.
- **Sin emojis:** Queda prohibido el uso de caracteres graficos o emojis en los mensajes de commit.
- **Trazabilidad vinculante con el Backlog:** Cada commit asociado a una historia de usuario debe finalizar obligatoriamente con el codigo del requerimiento entre corchetes (ej. `[#RF-01.01]`).

Tabla 4.1: Tipos estandar de commit y su aplicabilidad

Prefijo	Significado Semantico	Ejemplo Aplicado al Proyecto
`feat`	Incorporacion de una nueva funcionalidad	`feat(auth): crear modelo user con hashing bcrypt [#RF-01.01]`
`fix`	Correccion de un defecto o fallo funcional	`fix(daemon): resolver falso positivo por timeout [#RF-05.02]`
`perf`	Optimizacion de rendimiento o consumo	`perf(db): agregar indice compuesto en eventos [#RF-06.02]`
`refactor`	Reestructuracion sin alterar el comportamiento	`refactor(api): modularizar controladores de red [#RNF-12]`
`style`	Ajustes de formato visual, espaciado o CSS	`style(ui): normalizar badges de disponibilidad [#RNF-11]`
`docs`	Modificaciones en documentacion tecnica	`docs(metaphors): actualizar estandar de flujo git`
`test`	Incorporacion o correccion de pruebas	`test(auth): anadir pruebas de bloqueo de intentos [#RF-01.07]`
`chore`	Mantenimiento de paquetes, build o CI/CD	`chore(deps): actualizar dependencias de express y vite`

Capítulo 5

Flujo de Trabajo Paso a Paso y Comandos Operativos

A continuacion se presenta la guia operacional que todo desarrollador debe ejecutar en su terminal de Linux para trabajar colaborativamente de manera disciplinada y segura:

5.1 Paso 1: Configuracion Inicial de la Identidad Local

Antes de comenzar a trabajar en cualquier estacion de desarrollo, el desarrollador configura su identidad institucional y las opciones globales de merge:

```
# Configuracion de identidad del desarrollador
git config --global user.name "Nombre␣del␣Desarrollador"
git config --global user.email "usuario@espoch.edu.ec"

# Configuracion recomendada de integracion y saltos de linea
git config --global pull.rebase true
git config --global core.autocrlf input
git config --global init.defaultBranch main
```

5.2 Paso 2: Inicio de una Nueva Caracteristica (Branching)

Toda nueva tarea parte siempre desde la version mas reciente de la rama `develop`:

```
git checkout develop
git pull origin develop
git checkout -b feature/auth-user-model
```

5.3 Paso 3: Desarrollo Local y Commits Atomicos

El desarrollador implementa su solucion realizando commits pequenos, verificables y auto-contenidos:

```
git status
git add server/src/models/user.model.js
git commit -m "feat(auth):␣crear␣modelo␣user␣con␣hashing␣bcrypt␣[#
   RF-01.01]"
```

5.4 Paso 4: Sincronizacion Continua contra Develop

Para evitar acumulaciones masivas de conflictos, antes de publicar la rama se debe rebasar contra los ultimos cambios integrados por el equipo en `develop`:

```
git fetch origin
git rebase origin/develop
```

5.5 Paso 5: Publicacion de la Rama y Creacion del Pull Request

```
git push -u origin feature/auth-user-model
```

Una vez publicada la rama, el desarrollador ingresa a la plataforma web de Git (GitHub o GitLab) y abre un Pull Request dirigido estrictamente hacia la rama **`develop`**.

5.6 Paso 6: Plantilla Obligatoria de Pull Request (Code Review)

Todo Pull Request debe incluir en su descripcion la siguiente informacion estandarizada para facilitar la evaluacion del revisor:

```
### Descripcion del Cambio:
Implementacion desde cero del modelo User con hashing bcrypt de 12
rondas, validacion de roles institucionales y bloqueo de fuerza
   bruta.

### Requerimientos Asociados:
- RF-01.01 (Autenticacion de usuarios)
- RNF-05 (Seguridad criptografica de contrasenas)

### Verificacion y Pruebas Realizadas:
- [x] Compilacion y pruebas sintacticas ejecutadas exitosamente.
- [x] Verificado hash bcrypt con salt rounds = 12.
- [x] Verificada restriccion de unicidad en correo institucional.

### Revisor Asignado:
@rol-revisor (ej. Lider Tecnico / Desarrollador Backend)
```

5.7 Paso 7: Integracion y Limpieza de Ramas Locales

Una vez aprobado y fusionado el Pull Request en `develop`, el autor elimina su rama local para mantener limpio su entorno de trabajo:

```
git checkout develop
git pull origin develop
git branch -d feature/auth-user-model
```

Capítulo 6

Politicas de Proteccion de Ramas y Estrategias de Fusion

Para salvaguardar la integridad del repositorio y cumplir las buenas practicas de la ingenieria de software profesional, se definen politicas de proteccion mandatorias en el servidor Git remoto:

6.1 Politicas de Proteccion en Ramas Principales (`main` y `develop`)

- **Bloqueo de Pushes Directos:** Ningun miembro del equipo, independientemente de sus privilegios administrativos, tiene permitido ejecutar un `git push` directo hacia las ramas `main` o `develop`. Todo cambio debe ingresar exclusivamente mediante Pull Request.
- **Revision Obligatoria de Codigo (Code Review):** Todo Pull Request requiere de al menos una aprobacion formal (Review Approved) de otro integrante del equipo antes de que el servidor permita su integracion.
- **Bloqueo de Reescribir Historial (Force Push):** Las ramas `main` y `develop` tienen deshabilitado permanentemente el comando `git push -force`, asegurando que el historial oficial jamas pueda ser sobreescrito o destruido accidentalmente.
- **Comprobacion de Compilacion Limpia:** Antes de fusionar, el codigo debe compilar sin errores en el cliente y en el servidor, garantizando que el tronco de desarrollo nunca quede en estado roto.

6.2 Estrategias Estandar de Fusion en Pull Requests

Al momento de aprobar e integrar un Pull Request en GitHub/GitLab, se aplican las siguientes estrategias diferenciadas segun el destino:

1. **Hacia la rama `develop` (Integracion de Caracteristicas):**
   - Se utiliza preferentemente **Squash and Merge** o **Rebase and Merge**.
   - **Objetivo:** Condensar los commits intermedios de trabajo local en un unico commit coherente y autocontenido sobre `develop`, manteniendo el arbol de desarrollo lineal, limpio y facilmente legible.
2. **Hacia la rama `main` (Cierre de Release de Sprint):**
   - Se utiliza estrictamente **Merge Commit explicito** (`-no-ff`).
   - **Objetivo:** Dejar constancia formal del hito de integracion entre `develop` y `main`, sirviendo de anclaje para la colocacion de la etiqueta de version semantica (*Git Tag*).

Capítulo 7

Resolucion de Conflictos y Tecnicas Operativas Avanzadas

7.1 Protocolo Estandar de Resolucion de Conflictos

Cuando dos desarrolladores modifican lineas contiguas de un mismo archivo, Git detiene el proceso de rebase o merge y marca los bloques en conflicto. El desarrollador debe seguir este protocolo estructurado:

1. **Identificar los archivos afectados:** Ejecutar `git status` para listar los ficheros marcados como *both modified*.
2. **Inspeccionar los marcadores de conflicto:** Abrir el archivo y localizar las etiquetas insertadas por Git:

```
<<<<<<< HEAD (cambios provenientes de develop)
const PING_TIMEOUT = 3000;
=======
const PING_TIMEOUT = 5000; // Cambio local de la rama
>>>>>>> feature/ajuste-timeout
```

3. **Resolver y conciliar el codigo:** En acuerdo con el companero autor del cambio en conflicto, se deja la version tecnicamente correcta y se eliminan los marcadores «««<, ======= y »»»>.
4. **Verificar compilacion y pruebas:** Comprobar que el proyecto compila y no se introdujeron errores de sintaxis tras la conciliacion.
5. **Continuar el proceso de rebase:**

```
git add <archivo-corregido>
git rebase --continue
```

6. **Abortar en caso de bloqueo critico:** Si el desarrollador se percata de que la conciliacion es erronea o compleja de resolver en caliente, puede abortar el proceso sin alterar su rama:

```
git rebase --abort
```

7.2 Gestion de Trabajo en Progreso con `git stash`

Cuando un desarrollador esta trabajando en una rama y debe cambiar urgentemente de contexto o asistir a otro modulo sin confirmar cambios incompletos, utiliza el almacen temporal (*Stash*):

```
# Guardar cambios sin confirmar con mensaje descriptivo
git stash push -m "avance␣temporal␣de␣controladores␣de␣red"

# Consultar la lista de cambios guardados en el stash
git stash list

# Recuperar y aplicar los ultimos cambios guardados
git stash pop

# Descartar los cambios guardados si ya no son necesarios
git stash drop
```

7.3 Deshacer Cambios con Seguridad: `git revert` frente a `git reset`

La gestion segura de errores depende del ambito de la rama:

- **Ramas publicas compartidas (`main`, `develop`):** Se utiliza exclusivamente **`git revert <commit-hash>`**. Este comando genera un nuevo commit que aplica la operacion inversa exacta del cambio defectuoso, preservando intacto el historial anterior y evitando discrepancias entre los repositorios locales del equipo.
- **Ramas privadas de trabajo (`feature/*`):** Antes de publicar la rama al servidor remoto, el desarrollador puede deshacer su ultimo commit local conservando las modificaciones en el directorio de trabajo mediante:

  ```
  git reset --soft HEAD~1
  ```

- **Regla de oro de seguridad:** Queda terminantemente prohibido ejecutar `git reset --hard` sobre ramas publicas o compartidas.

Capítulo 8

Versionado Semantico y Gestion de Etiquetas (Git Tagging)

8.1 Estandar SemVer 2.0.0

Para el control de versiones en la rama `main`, el proyecto se acoge al estandar internacional **Semantic Versioning (SemVer 2.0.0)**, estructurado bajo el formato:

`vMAJOR.MINOR.PATCH`

- **MAJOR (Version Mayor):** Se incrementa ante cambios estructurales mayores o modificaciones que rompen compatibilidad hacia atras (ejemplo: `v2.0.0` al cambiar la arquitectura del sistema).
- **MINOR (Version Menor):** Se incrementa al incorporar nuevas funcionalidades estables que no rompen compatibilidad, correspondiendo tipicamente a los cierres de cada sprint evaluable (ejemplo: `v1.1.0` tras culminar el Sprint 2).
- **PATCH (Parche):** Se incrementa al aplicar correcciones de errores urgentes introducidas mediante ramas de `hotfix` (ejemplo: `v1.0.1`).

8.2 Comandos de Creacion y Gestion de Etiquetas

Al culminar un release en `main`, el responsable de integracion genera una etiqueta anotada (*Annotated Tag*):

```
# Crear una etiqueta anotada con mensaje descriptivo
git tag -a v1.0.0 -m "Release␣Oficial␣Sprint␣1:␣Modulo␣de␣
   Autenticacion␣y␣Usuarios"

# Publicar la etiqueta en el repositorio remoto
git push origin v1.0.0

# Listar las etiquetas existentes en el proyecto
git tag -l

# Inspeccionar los detalles y metadatos de una etiqueta
git show v1.0.0
```

Capítulo 9

Automatizacion con Git Hooks y Diagnostico del Repositorio

9.1 Control Preventivo de Calidad con Git Hooks

Para asegurar que ningun commit incumpla las directrices de formateo o rompa la sintaxis antes de llegar al servidor remoto, se configuran ganchos locales automatizados (*Git Hooks*) mediante herramientas como Husky:

- **Gancho `pre-commit`:** Se dispara automaticamente antes de registrar el commit. Ejecuta la verificacion de sintaxis y linter en el cliente y servidor, abortando el commit si existen errores de codigo.
- **Gancho `commit-msg`:** Valida mediante una expresion regular que el mensaje introducido por el desarrollador cumpla la estructura estricta de Conventional Commits y contenga el identificador del requerimiento asociado (`[#RF-XX.XX]`).

9.2 Comandos de Diagnostico e Inspeccion del Historial

Para analizar el estado del repositorio y auditar la trazabilidad del codigo, todo integrante del equipo dispone de los siguientes comandos de diagnostico:

```
# Visualizar el arbol grafico de ramas y commits de forma
   condensada
git log --graph --oneline --decorate --all

# Analizar las lineas modificadas en el area de trabajo
git diff --stat

# Identificar al autor y commit exacto de cada linea de un archivo
   (auditoria)
git blame server/src/app.js

# Consultar el registro de movimientos del puntero HEAD (
   recuperacion de emergencia)
git reflog
```

Capítulo 10

Seguridad del Repositorio y Gestion de Archivos Ignorados

10.1 Regla de Oro: Cero Secretos en el Repositorio

Bajo las directrices de seguridad de la informacion ISO/IEC 27001, esta terminantemente prohibido versionar credenciales, cadenas de conexion a bases de datos con claves de usuario, claves privadas de cifrado o tokens secretos en el control de versiones.

10.2 Archivo `.gitignore` Obligatorio

En la raiz del proyecto se mantiene un archivo `.gitignore` exhaustivo con la siguiente configuracion minima obligatoria:

```
# Dependencias de paquetes y modulos
node_modules/
.pnp
.pnp.js

# Artefactos de compilacion y distribucion
dist/
client/dist/
build/

# Archivos de entorno y secretos criptograficos
.env
.env.local
.env.production
server/.env
client/.env
*.pem
*.key

# Logs del sistema y herramientas de depuracion
logs/
*.log
npm-debug.log*
yarn-debug.log*
```

```
# Archivos temporales de compiladores LaTeX
*.aux
*.lot
*.toc
*.log
*.out
*.synctex.gz
*.fls
*.fdb_latexmk

# Archivos temporales de sistemas operativos y editores
.DS_Store
Thumbs.db
.vscode/
.idea/
*.sw?

# Reportes de pruebas y cobertura
coverage/
.nyc_output/
test-results/
playwright-report/

# Directorios temporales
tmp/
temp/
```

10.3 Plantilla de Variables de Entorno (`.env.example`)

Para que cualquier desarrollador pueda configurar su entorno local sin comprometer secretos, se versiona publicamente el archivo `server/.env.example` con valores ficticios:

```
# Configuracion del Servidor API
PORT=5000
NODE_ENV=development

# Base de Datos MongoDB
MONGODB_URI=mongodb://localhost:27017/network_monitoring

# Seguridad Criptografica JWT
JWT_SECRET=super_secret_jwt_access_key_espoch_noc_2026
JWT_REFRESH_SECRET=super_secret_jwt_refresh_key_espoch_noc_2026
JWT_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=7d
```

```
# Origen CORS Autorizado
CORS_ORIGIN=http://localhost:5173

# Limites Seguros de Monitoreo (Guardrails Seccion 8.7)
PING_INTERVAL_SEC=30
DISCOVERY_INTERVAL_MIN=10
PING_TIMEOUT_MS=3000
CONSECUTIVE_FAILURES_FOR_DOWN=3
CONSECUTIVE_SUCCESSES_FOR_UP=2
MAX_CONCURRENT_PINGS=15
SWITCH_QUORUM_THRESHOLD=0.70
```

Capítulo 11

Conclusiones

La disciplina en el control de versiones constituye el cimiento metodologico que asegura la integridad, la reproducibilidad y el progreso ordenado del sistema de monitoreo de redes de la ESPOCH.

La implementacion rigurosa de este estandar de Git -basado en GitFlow, Conventional Commits, revisiones obligatorias de codigo, politicas de proteccion de ramas y tecnicas formales de resolucion de conflictos- permite que cualquier desarrollador que se sume al proyecto comprenda de inmediato como colaborar eficazmente, preservando la calidad profesional del software.